



南開大學
Nankai University

数据结构实验报告十

图

学号：2412266

姓名：杨滢

专业：信息安全

学院：密码与网络空间安全学院

目录

1 题目一	2
1.1 题目表述	2
1.2 代码 1 的测试用例	2
1.3 思路	3
1.3.1 输入解析模块	3
1.3.2 图数据结构设计	3
1.3.3 BFS 核心算法	3
1.3.4 边界情况处理	4
1.4 最短路径正确性证明	4
1.5 复杂度分析	5
1.6 代码	5
1.7 测试用例截图	10
2 题目二	11
2.1 题目表述	11
2.2 代码 2 的测试用例	11
2.3 思路	11
2.3.1 输入解析模块	11
2.3.2 图数据结构设计	11
2.3.3 核心算法设计	12
2.4 复杂度分析	12
2.5 代码	12
2.6 测试用例截图	18

1 题目一

1.1 题目表述

使用 BFS 而不是 DFS, 编写程序 16-8 的另一个版本。证明由这个版本所得到的从 theSource 到 theDestination 的路径是最短路径。

程序 16-8:

```
1 int* findPath(int theSource, int theDestination)
2 {
3     // 寻找一条从顶点 theSource 到顶点 theDestination 的路径
4     // 返回一个数组 path, 从索引 1 开始表示路径。path[0] 表示路径长度
5     // 如果路径不存在, 返回 NULL
6     // 为寻找路径的递归算法初始化
7     int n = numberOfVertices();
8     path = new int [n + 1];
9     path[1] = theSource; // 第一个顶点总是
10    length = 1;          // 当前路径长度 + 1
11    destination = theDestination;
12    reach = new int [n + 1];
13    for (int i = 1; i <= n; i++)
14        reach[i] = 0;
15    // 搜索路径
16    if (theSource == theDestination || rFindPath(theSource))
17    {
18        // 找到一条路径
19        path[0] = length - 1;
20    }
21    else
22    {
23        delete [] path;
24        path = NULL;
25    }
26    delete [] reach;
27    return path;
28 }
```

1.2 代码 1 的测试用例

1. {1}, {}, findPath(G0, 1, 1)
2. {1,2,3}, {(1,2)}, findPath(G, 1, 3)
3. {1,2,3,4,5}, {(1,2),(2,3),(3,4),(4,5)}, findPath(G, 1, 5)

4. $\{1,2,3,4,5\}$, $\{(1,2),(2,5),(1,3),(3,4),(4,5)\}$, $\text{findPath}(G, 1, 5)$
5. $\{1,2,3,4\}$, $\{(1,2),(2,4),(1,3),(3,4)\}$, $\text{findPath}(G, 1, 4)$

1.3 思路

BFS 版本 findPath 的实现思路主要分为以下几个关键步骤：

1.3.1 输入解析模块

- **顶点集解析**：处理形如 $\{1, 2, 3\}$ 的字符串
 1. 移除首尾的花括号
 2. 按逗号分割字符串
 3. 将每个子字符串转换为整数
- **边集解析**
 1. 处理空边集 $\{\}$ 的特殊情况
 2. 识别括号对 (u, v)
 3. 提取顶点编号 u 和 v
- **函数调用解析**：解析 $\text{findPath}(G, s, t)$ 调用
 1. 提取括号内的参数
 2. 跳过图名参数
 3. 获取源点 s 和目标点 t

1.3.2 图数据结构设计

采用邻接表表示图，原因如下：

- **空间效率高**：适合稀疏图，空间复杂度 $O(V + E)$
- **遍历效率高**：获取顶点的邻接点时间复杂度 $O(\deg(v))$
- **无向图处理**：每条边 (u, v) 在邻接表中双向添加

$$\text{adjList}[u] \leftarrow v, \quad \text{adjList}[v] \leftarrow u$$

1.3.3 BFS 核心算法

BFS（广度优先搜索）的核心思想是按层遍历图。使用数组模拟队列可以避免递归调用，同时保证搜索顺序是先进先出（FIFO）。

1. 初始化：

- 创建 `visited` 数组，标记顶点是否访问过
- 创建 `parent` 数组，记录每个顶点的前驱节点

- 创建 `queue` 数组作为队列，用 `front` 和 `rear` 指针管理

2. 起点入队：

- 将源点加入队列
- 标记源点为已访问

3. 循环搜索：

- 当队列不为空且未找到目标时：
 - (a) 从队头取出当前顶点 u
 - (b) 遍历 u 的所有邻接顶点 v
 - (c) 如果 v 未访问过：
 - 标记 v 为已访问
 - 记录 v 的前驱为 u
 - 将 v 加入队尾
 - 如果 v 是目标点，结束搜索

4. 路径重建：

- (a) **反向回溯**：从目标点 t 开始，通过 `parent` 数组回溯到源点 s

路径反向 = $[t, \text{parent}[t], \text{parent}[\text{parent}[t]], \dots, s]$

- (b) **反转路径**：将反向路径反转得到正向路径

正向路径 = $[s, \dots, t]$

- (c) **构建输出数组**：

- `path[0]`：路径长度（边数） L
- `path[1..L + 1]`：顶点序列 $s \rightarrow \dots \rightarrow t$

1.3.4 边界情况处理

- $s = t$ ：直接返回 $[s]$
- 不连通图：返回 `NULL`
- 空图/单顶点图：特殊处理
- 多条最短路径：BFS 返回首先发现的那条

1.4 最短路径正确性证明

定理：BFS 找到的从 s 到 t 的路径是最短路径。

证明：

1. **定义**：设 $d(v)$ 表示从 s 到 v 的最短距离（边数）

2. BFS 性质: BFS 按距离 $d(v)$ 递增的顺序访问顶点

先访问所有 $d(v) = k$ 的顶点, 再访问 $d(v) = k + 1$ 的顶点

3. 数学归纳法:

- 基础步: $d(s) = 0$, BFS 首先访问 s
- 归纳步: 假设所有 $d(v) \leq k$ 的顶点已按正确顺序访问
- 对于任意 $d(w) = k + 1$ 的顶点 w , 存在 u 使得 $(u, w) \in E$ 且 $d(u) = k$
- BFS 会在访问 u 时发现 w , 因此 $d(w) = k + 1$ 的顶点在 $d(v) = k$ 的顶点之后访问

4. 反证法:

- 假设存在更短路径长度 $L' < L$ (L 为 BFS 找到的路径长度)
- 则 $d(t) = L'$
- 由 BFS 性质, t 应在第 L' 层被访问
- 但 BFS 在第 L 层才访问 t , 矛盾
- 因此不存在比 BFS 找到的更短路径

1.5 复杂度分析

- 时间复杂度: $O(V + E)$
 - 每个顶点入队出队一次: $O(V)$
 - 每条边检查两次 (无向图): $O(2E) = O(E)$
- 空间复杂度: $O(V + E)$
 - 邻接表存储: $O(V + E)$
 - BFS 辅助数组: $O(V)$

1.6 代码

```
1 #include <iostream>
2 #include <fstream>
3 #include <sstream>
4 #include <vector>
5 #include <string>
6 #include <algorithm>
7 using namespace std;
8 vector<int> parseVertices(const string& line) {
9     vector<int> vertices;
10    string stripped = line.substr(1, line.length() - 2);
11    stringstream ss(stripped);
```

```
12     string token;
13     while (getline(ss, token, ',')) {
14         if (!token.empty())
15             vertices.push_back(stoi(token));
16     }
17     return vertices;
18 }
19 vector<pair<int, int>> parseEdges(const string& line) {
20     vector<pair<int, int>> edges;
21     if (line == "{}") return edges;
22     string stripped = line.substr(1, line.length() - 2);
23     stringstream ss(stripped);
24     string token;
25     while (getline(ss, token, '(')) {
26         if (token.empty()) continue;
27         size_t start = token.find_last_of('(');
28         if (start != string::npos) {
29             string pairStr = token.substr(start + 1);
30             size_t comma = pairStr.find(',');
31             if (comma != string::npos) {
32                 int u = stoi(pairStr.substr(0, comma));
33                 int v = stoi(pairStr.substr(comma + 1));
34                 edges.push_back({ u, v });
35             }
36         }
37     }
38     return edges;
39 }
40 pair<int, int> parseFindPathCall(const string& line) {
41     size_t start = line.find('(');
42     size_t end = line.find(')');
43     string args = line.substr(start + 1, end - start - 1);
44
45     stringstream ss(args);
46     string token;
47     getline(ss, token, ',');
48     getline(ss, token, ',');
49     int source = stoi(token);
50     getline(ss, token, ',');
51     int dest = stoi(token);
52     return { source, dest };
53 }
```

```
54 class Graph {
55 private:
56     vector<int> vertices;
57     vector<vector<int>> adjList;
58     int maxVertex;
59 public:
60     Graph(const vector<int>& verts, const vector<pair<int, int>>& edges) :
        vertices(verts) {
61         maxVertex = 0;
62         for (int v : verts) {
63             if (v > maxVertex) maxVertex = v;
64         }
65         adjList.resize(maxVertex + 1);
66
67         for (const auto& edge : edges) {
68             int u = edge.first;
69             int v = edge.second;
70             adjList[u].push_back(v);
71             adjList[v].push_back(u);
72         }
73     }
74     int numberOfVertices() const {
75         return vertices.size();
76     }
77     vector<int> getAdjacentVertices(int v) const {
78         if (v <= 0 || v > maxVertex) return {};
79         return adjList[v];
80     }
81     int getMaxVertex() const {
82         return maxVertex;
83     }
84 };
85 int* findPath(int theSource, int theDestination, Graph& graph) {
86     if (theSource == theDestination) {
87         int* path = new int[2];
88         path[0] = 0;
89         path[1] = theSource;
90         return path;
91     }
92     int maxVertex = graph.getMaxVertex();
93     int arraySize = maxVertex + 1;
```

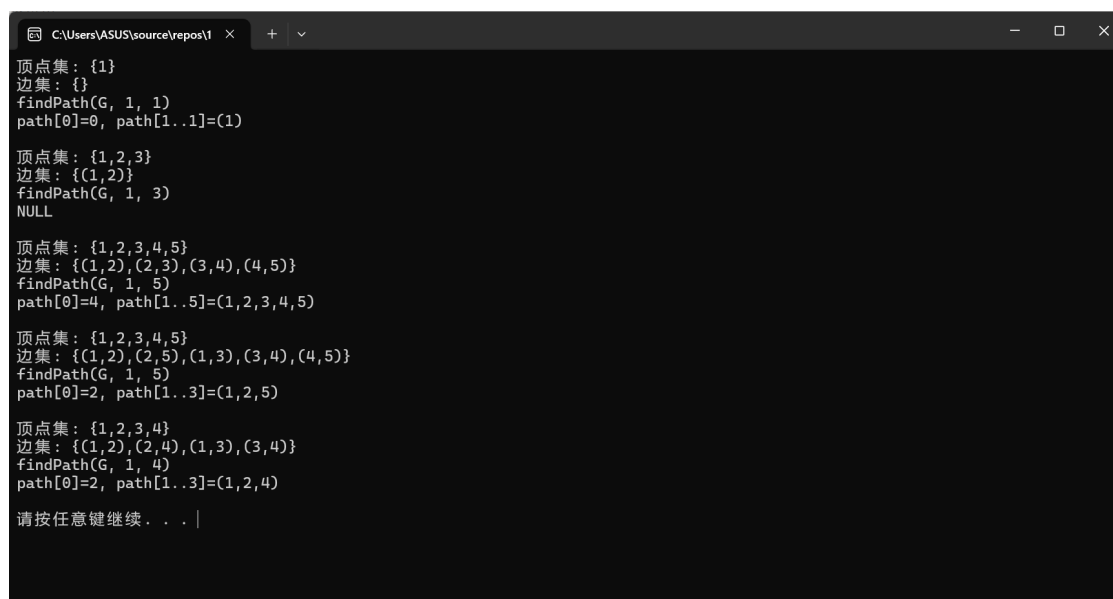


```
95     bool* visited = new bool[arraySize]();
96     int* parent = new int[arraySize];
97     int* queue = new int[arraySize];
98
99     for (int i = 0; i < arraySize; i++) {
100         visited[i] = false;
101         parent[i] = -1;
102     }
103     int front = 0, rear = 0;
104     queue[rear++] = theSource;
105     visited[theSource] = true;
106     bool found = false;
107     while (front < rear && !found) {
108         int current = queue[front++];
109
110         vector<int> neighbors = graph.getAdjacentVertices(current);
111
112         for (int neighbor : neighbors) {
113             if (!visited[neighbor]) {
114                 visited[neighbor] = true;
115                 parent[neighbor] = current;
116                 queue[rear++] = neighbor;
117
118                 if (neighbor == theDestination) {
119                     found = true;
120                     break;
121                 }
122             }
123         }
124     }
125     int* path = nullptr;
126     if (found) {
127         vector<int> tempPath;
128         int node = theDestination;
129         while (node != -1) {
130             tempPath.push_back(node);
131             node = parent[node];
132         }
133         reverse(tempPath.begin(), tempPath.end());
134         path = new int[tempPath.size() + 1];
135         path[0] = tempPath.size() - 1;
136         for (size_t i = 0; i < tempPath.size(); i++) {
```

```
137     path[i + 1] = tempPath[i];
138 }
139 }
140 delete[] visited;
141 delete[] parent;
142 delete[] queue;
143 return path;
144 }
145 void printPath(int* path) {
146     if (path == nullptr) {
147         cout << "NULL" << endl;
148         return;
149     }
150     int length = path[0];
151     cout << "path[0]=" << length << ", ";
152     cout << "path[1.." << (length + 1) << "]=(";
153     for (int i = 1; i <= length + 1; i++) {
154         cout << path[i];
155         if (i < length + 1) cout << ",";
156     }
157     cout << ")" << endl;
158 }
159 int main() {
160     ifstream infile("input.txt");
161     if (!infile) {
162         cerr << "WRONG" << endl;
163         return 1;
164     }
165     string line;
166     int testCaseNum = 0;
167     while (getline(infile, line)) {
168         if (line.empty()) continue;
169         testCaseNum++;
170
171         vector<int> vertices = parseVertices(line);
172         cout << "顶点集: {";
173         for (size_t i = 0; i < vertices.size(); i++) {
174             cout << vertices[i];
175             if (i < vertices.size() - 1) cout << ",";
176         }
177         cout << "}" << endl;
178         getline(infile, line);
```

```
179     vector<pair<int, int>> edges = parseEdges(line);
180     cout << "边集: {" ;
181     for (size_t i = 0; i < edges.size(); i++) {
182         cout << "(" << edges[i].first << ", " << edges[i].second << ")";
183         if (i < edges.size() - 1) cout << ",";
184     }
185     cout << "}" << endl;
186     getline(infile, line);
187     pair<int, int> endpoints = parseFindPathCall(line);
188     int source = endpoints.first;
189     int dest = endpoints.second;
190     cout << "findPath(G, " << source << ", " << dest << ")" << endl;
191     Graph g(vertices, edges);
192     int* path = findPath(source, dest, g);
193     printPath(path);
194     if (path != nullptr) {
195         delete[] path;
196     }
197     cout << endl;
198 }
199 infile.close();
200 system("pause");
201 return 0;
202 }
```

1.7 测试用例截图



```
C:\Users\ASUS\source\repos\T
顶点集: {1}
边集: {}
findPath(G, 1, 1)
path[0]=0, path[1..1]=(1)

顶点集: {1,2,3}
边集: {(1,2)}
findPath(G, 1, 3)
NULL

顶点集: {1,2,3,4,5}
边集: {(1,2),(2,3),(3,4),(4,5)}
findPath(G, 1, 5)
path[0]=4, path[1..5]=(1,2,3,4,5)

顶点集: {1,2,3,4,5}
边集: {(1,2),(2,5),(1,3),(3,4),(4,5)}
findPath(G, 1, 5)
path[0]=2, path[1..3]=(1,2,5)

顶点集: {1,2,3,4}
边集: {(1,2),(2,4),(1,3),(3,4)}
findPath(G, 1, 4)
path[0]=2, path[1..3]=(1,2,4)

请按任意键继续 . . .
```

2 题目二

2.1 题目表述

设 G 是一个无向图。它的传递闭包 (transitive closure) 是一个 01 数组 tc , 当且仅当 G 存在一条边数大于 1 的从 i 到 j 的路径时, $tc[i][j]=1$ 。编写一个方法 `graph::undirectedTC()`, 计算且返回 G 的传递闭包。方法的复杂性应为 $O(n^3)$, 其中 n 是 G 的顶点数。(提示: 采用构件标记策略。)

2.2 代码 2 的测试用例

1. $V=\{1,2,3\}$, $E=null$, $tc = \text{undirectedTC}()$
2. $V=\{1,2\}$, $E=\{(1,2)\}$, $tc = \text{undirectedTC}()$
3. $V=\{1,2,3\}$, $E=\{(1,2),(2,3)\}$, $tc = \text{undirectedTC}()$
4. $V=\{1,2,3,4\}$, $E=\{(1,2),(1,3),(1,4)\}$, $tc = \text{undirectedTC}()$
5. $V=\{1,2,3\}$, $E=\{(1,2),(2,3),(1,3)\}$, $tc = \text{undirectedTC}()$

2.3 思路

2.3.1 输入解析模块

解析流程:

1. 顶点集解析:

- 定位花括号 $\{\}$
- 提取括号内内容
- 按逗号分割
- 转换为整数并排序

2. 边集解析:

- 处理特殊标记: $E=null$ 、 $E=\{\}$
- 定位花括号和括号对
- 提取顶点对 (u, v)

2.3.2 图数据结构设计

- 顶点映射: 使用 `vertex_to_index` 数组实现顶点编号到数组索引的快速映射

$$\text{getIndex}(v) = \begin{cases} \text{index} & \text{if } 0 \leq v < |\text{vertex_to_index}| \\ -1 & \text{otherwise} \end{cases}$$

- 邻接表: `adj` 存储每个顶点的邻居列表
- 传递闭包矩阵: `tc` 存储最终结果

2.3.3 核心算法设计

采用 **逐对顶点验证法**：

1. **遍历所有顶点对**：对于每个顶点对 (i, j)

- 如果 $i = j$ ： $tc[i][j] = 0$ （对角线）
- 否则：调用 `hasPathLongerThanOne(i, j)`

2. **长度 >1 路径检查算法**：

- **输入**：源点索引 `src_idx`，目标点索引 `dst_idx`
- **输出**：是否存在长度 > 1 的路径
- **步骤**：
 - (a) 检查源点和目标点是否直接相连
 - (b) 使用 **BFS (广度优先搜索)** 寻找路径
 - (c) 关键约束：排除直接边（长度 =1 的路径）

2.4 复杂度分析

- **时间复杂度**：

- 对每对顶点调用一次 BFS： $O(n^2)$
- 每次 BFS： $O(V + E)$
- 总复杂度： $O(n^2 \cdot (V + E))$

- **空间复杂度**：

- 邻接表： $O(V + E)$
- 传递闭包矩阵： $O(n^2)$
- BFS 队列和访问数组： $O(V)$

2.5 代码

```
1 #include <iostream>
2 #include <fstream>
3 #include <sstream>
4 #include <vector>
5 #include <string>
6 #include <cstring>
7 #include <algorithm>
8 #include <queue>
9 #include <utility>
10 using namespace std;
11 class Graph {
```

```
12 private:
13     int n;
14     vector<vector<int>> adj;
15     vector<vector<bool>> tc;
16     vector<int> vertices;
17     vector<int> vertex_to_index;
18     int getIndex(int v) const {
19         if (v >= 0 && v < (int)vertex_to_index.size()) {
20             return vertex_to_index[v];
21         }
22         return -1;
23     }
24     bool hasPathLongerThanOne(int src_idx, int dst_idx) {
25         if (src_idx == dst_idx) return false;
26         int src = vertices[src_idx];
27         int dst = vertices[dst_idx];
28         bool directlyConnected = false;
29         for (int neighbor : adj[src_idx]) {
30             if (neighbor == dst) {
31                 directlyConnected = true;
32                 break;
33             }
34         }
35
36         queue<pair<int, int>> q;
37         vector<bool> visited(n, false);
38
39         visited[src_idx] = true;
40
41         for (int neighbor_val : adj[src_idx]) {
42             int neighbor_idx = getIndex(neighbor_val);
43             if (neighbor_idx != -1) {
44                 if (neighbor_idx == dst_idx) {
45                     if (directlyConnected) {
46                         continue;
47                     }
48                     else {
49                         continue;
50                     }
51                 }
52                 q.push(make_pair(neighbor_idx, 1));
53                 visited[neighbor_idx] = true;
```

```
54     }
55 }
56
57 while (!q.empty()) {
58     pair<int, int> current = q.front();
59     q.pop();
60
61     int current_idx = current.first;
62     int current_length = current.second;
63
64     if (current_idx == dst_idx) {
65         return true;
66     }
67
68     for (int neighbor_val : adj[current_idx]) {
69         int neighbor_idx = getIndex(neighbor_val);
70         if (neighbor_idx != -1 && !visited[neighbor_idx]) {
71             q.push(make_pair(neighbor_idx, current_length + 1));
72             visited[neighbor_idx] = true;
73         }
74     }
75 }
76
77 return false;
78 }
79 public:
80     Graph(const vector<int>& verts, const vector<pair<int, int>>& edges) :
81         vertices(verts) {
82         sort(vertices.begin(), vertices.end());
83         n = (int)vertices.size();
84         int max_vertex = 0;
85         for (int v : vertices) {
86             if (v > max_vertex) max_vertex = v;
87         }
88         vertex_to_index.resize(max_vertex + 1, -1);
89         for (int i = 0; i < n; i++) {
90             vertex_to_index[vertices[i]] = i;
91         }
92         adj.resize(n);
93         tc.resize(n, vector<bool>(n, false));
94         for (const auto& edge : edges) {
95             int u = edge.first;
```

```
95         int v = edge.second;
96
97         int u_idx = getIndex(u);
98         int v_idx = getIndex(v);
99
100         if (u_idx != -1 && v_idx != -1) {
101             adj[u_idx].push_back(v);
102             adj[v_idx].push_back(u);
103         }
104     }
105 }
106 vector<vector<bool>> undirectedTC() {
107     for (int i = 0; i < n; i++) {
108         for (int j = 0; j < n; j++) {
109             if (i == j) {
110                 tc[i][j] = false;
111             }
112             else {
113                 tc[i][j] = hasPathLongerThanOne(i, j);
114             }
115         }
116     }
117
118     return tc;
119 }
120 void printTC() {
121     for (int i = 0; i < n; i++) {
122         cout << "i=" << vertices[i] << ": (";
123         for (int j = 0; j < n; j++) {
124             cout << (tc[i][j] ? "1" : "0");
125             if (j < n - 1) cout << ",";
126         }
127         cout << ")" << endl;
128     }
129 }
130 };
131
132 static vector<int> parseVertices(const string& line) {
133     vector<int> vertices;
134     size_t start = line.find('{');
135     size_t end = line.rfind('}');
136     if (start != string::npos && end != string::npos && start < end) {
```



```
137     string content = line.substr(start + 1, end - start - 1);
138     content.erase(remove(content.begin(), content.end(), ' '), content.end());
139
140     if (!content.empty()) {
141         stringstream ss(content);
142         string token;
143
144         while (getline(ss, token, ',')) {
145             if (!token.empty()) {
146                 vertices.push_back(stoi(token));
147             }
148         }
149     }
150 }
151
152 sort(vertices.begin(), vertices.end());
153 return vertices;
154 }
155
156 static vector<pair<int, int>> parseEdges(const string& line) {
157     vector<pair<int, int>> edges;
158     if (line.find("E=null") != string::npos) {
159         return edges;
160     }
161     if (line.find("E=") != string::npos) {
162         string e_part = line.substr(line.find("E=") + 2);
163         e_part.erase(remove(e_part.begin(), e_part.end(), ' '), e_part.end());
164         if (e_part == "{}" || e_part.find("{}") == 0) {
165             return edges;
166         }
167     }
168     size_t start = line.find('{');
169     size_t end = line.rfind('}');
170     if (start != string::npos && end != string::npos && start < end) {
171         string content = line.substr(start + 1, end - start - 1);
172         content.erase(remove(content.begin(), content.end(), ' '), content.end());
173
174         size_t pos = 0;
175         while (pos < content.length()) {
176             size_t open_paren = content.find('(', pos);
177             if (open_paren == string::npos) break;
```

```
179     size_t close_paren = content.find(')', open_paren);
180     if (close_paren == string::npos) break;
181
182     string pair_str = content.substr(open_paren + 1, close_paren - open_paren
183                                     - 1);
184     size_t comma = pair_str.find(',');
185
186     if (comma != string::npos) {
187         string u_str = pair_str.substr(0, comma);
188         string v_str = pair_str.substr(comma + 1);
189
190         if (!u_str.empty() && !v_str.empty()) {
191             int u = stoi(u_str);
192             int v = stoi(v_str);
193             edges.push_back(make_pair(u, v));
194         }
195     }
196
197     pos = close_paren + 1;
198 }
199
200 return edges;
201 }
202
203 int main() {
204     ifstream infile("input.txt");
205     if (!infile.is_open()) {
206         cerr << "WRONG" << endl;
207         return 1;
208     }
209     string line;
210     int testCase = 0;
211     while (getline(infile, line)) {
212         if (line.empty()) continue;
213
214         if (line.find("V={") == 0 || line.find("V = {") == 0) {
215             testCase++;
216             cout << "测试用例" << testCase << endl;
217             vector<int> vertices = parseVertices(line);
218             if (vertices.empty()) {
219                 cout << "WRONG" << endl;
```

```
220         continue;
221     }
222
223     vector<pair<int, int>> edges;
224     if (getline(infile, line)) {
225         edges = parseEdges(line);
226     }
227
228     if (getline(infile, line)) {
229     }
230
231     Graph g(vertices, edges);
232     g.undirectedTC();
233
234     g.printTC();
235     cout << endl;
236 }
237 }
238 infile.close();
239 system("pause");
240 return 0;
241 }
```

2.6 测试用例截图

